

CURSO 2: ALGORITMOS AVANZADOS Y ESTRUCTURAS DE DATOS

CLASE 05 • NIVEL 2 - INTERMEDIO

# Clase 05: Algoritmos de Ordenamiento: QuickSort y MergeSort

«Ordenar Barajas de Cartas con Divide y Vencerás»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

**Autor & Mentor:** William Rodríguez (Wisrovi)

**Rol:** AI Solutions Architect & Principal Software Engineer

**Python:** 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

## Acerca del Autor y Mentor



### William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



**GitHub:** [github.com/wisrovi](https://github.com/wisrovi)



**LinkedIn:** [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



**DockerHub:** [hub.docker.com/u/wisrovi](https://hub.docker.com/u/wisrovi)



**Website:** [wisrovi.dev](https://wisrovi.dev)

## Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



### El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

## Presentación de la Sesión

Bienvenido/a a la **CLASE 05** del **Curso 2: Algoritmos Avanzados y Estructuras de Datos**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

### Objetivos de la Sesión

**Competencia Conceptual:** Comprender la estabilidad, complejidad  $O(n \log n)$  y particionamiento por pivote.

**Competencia Práctica:** Implementar QuickSort y comprender el funcionamiento del algoritmo Timsort de Python.

## Estructura del Documento

| Sección             | Contenido Temático                  | Objetivo Pedagógico              |
|---------------------|-------------------------------------|----------------------------------|
| Pág 4: Fundamento   | Teoría, Modelo Mental y Metáfora    | Comprensión del concepto         |
| Pág 5: Flujo        | Diagrama de Arquitectura de Memoria | Visualización del flujo interno  |
| Pág 6: Código       | Implementación en Python 3.10+      | Patrones idiomáticos (PEP 8)     |
| Pág 7: Gotchas      | Antipatrones y Trampas Comunes      | Prevención de bugs en producción |
| Pág 8: Conclusiones | Cierre de Sesión & Notas de Repaso  | Consolidación del aprendizaje    |
| Pág 9: Bibliografía | Fuentes Canónicas y Desafío         | Ampliación y autoestudio         |

## Clase 05: Algoritmos de Ordenamiento: QuickSort y MergeSort

El ordenamiento es la base de la optimización en ciencias de la computación.

### ☀ Metáfora Central: Ordenar Barajas de Cartas con Divide y Vencerás

QuickSort elige un elemento pivote y separa las cartas en dos montones: menores a la izquierda, mayores a la derecha.

### Principios Teóricos y Modelo Mental

Algoritmos cuadráticos  $O(n^2)$  como Bubble o Insertion Sort son lentos para grandes volúmenes.

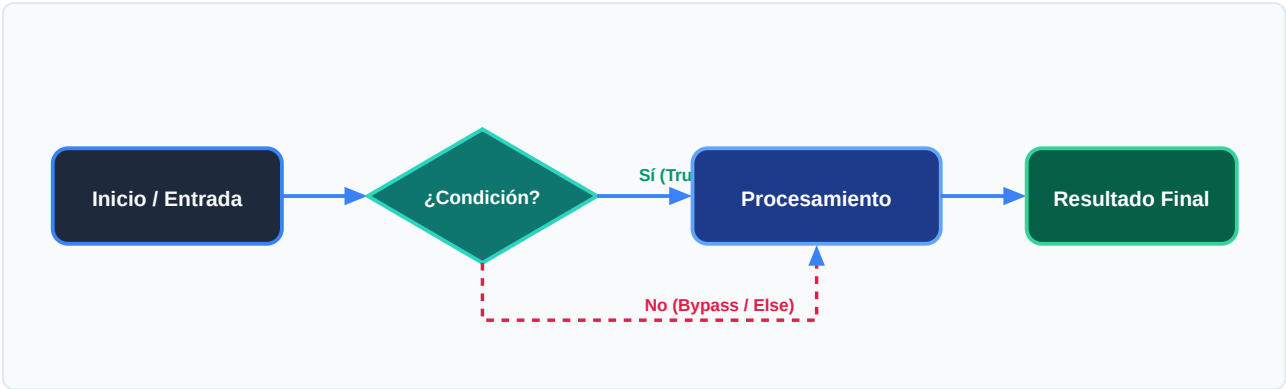
QuickSort y MergeSort tienen complejidad promedio  $O(n \log n)$ .

### ⚡ Regla de Oro en Python

En producción, usa siempre el método `.sort()` de Python, que implementa Timsort (híbrido optimizado en C).

## Arquitectura y Movimiento de Datos en Memoria

Particionamiento recursivo alrededor de un pivote.



### Desglose Paso a Paso del Diagrama

| Fase del Flujo      | Acción del Intérprete                                   | Estado en Memoria         |
|---------------------|---------------------------------------------------------|---------------------------|
| 1. Inicialización   | Caso base: $\text{len}(\text{arr}) \leq 1$ retorna arr. | Sublista trivial.         |
| 2. Evaluación       | Elección de pivote (elemento central).                  | Pivote seleccionado.      |
| 3. Transformación   | Partición: sublistas menores, iguales y mayores.        | Tres arreglos en memoria. |
| 4. Retorno / Salida | Llamadas recursivas y concatenación.                    | Lista combinada ordenada. |

#### Visualización Mental

Imagina partir un mazo de cartas por el centro hasta tener cartas individuales.

## Código de Demostración en Python 3.10+

Implementación idiomática de QuickSort:

main.py (Python 3.10+)

PEP 8 Compliant

```
def quicksort(arr: list[int]) -> list[int]:
    if len(arr) <= 1:
        return arr
    pivote = arr[len(arr) // 2]
    menores = [x for x in arr if x < pivote]
    iguales = [x for x in arr if x == pivote]
    mayores = [x for x in arr if x > pivote]
    return quicksort(menores) + iguales + quicksort(mayores)

desordenados = [38, 27, 43, 3, 9, 82, 10]
print("Ordenados:", quicksort(desordenados))
```

### Análisis de Ingeniería del Código

Recursión divide y vencerás con list comprehensions para particionado legible.



#### Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

## Trampas Frecuentes de Depuración (Gotchas)

El peor caso de QuickSort  $O(n^2)$ .

### ⚠ Gotcha Frecuente (Trampa de Principiante)

Elegir siempre el primer elemento como pivote en una lista que ya está ordenada genera  $O(n^2)$ .

## Comparativa: Antipatrón vs Patrón Recomendado

### ❌ Antipatrón

`pivote = arr[0]` # ❌ Degrada a  $O(n^2)$  si la lista ya viene ordenada

### ✅ Patrón Correcto

`pivote = arr[len(arr) // 2]` # ✅ Pivote central o aleatorio



### Consejo de Resiliencia en Producción

`sorted(iterable, key=lambda x: x.propiedad)` permite ordenar por cualquier criterio.

## Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 05: Algoritmos de Ordenamiento: QuickSort y MergeSort**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.



### Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

## Notas de Repaso Rápido

| Concepto            | Punto Clave a Recordar                                                                                                 |
|---------------------|------------------------------------------------------------------------------------------------------------------------|
| 1. Modelo Mental    | Ordenar Barajas de Cartas con Divide y Vencerás                                                                        |
| 2. Regla de Oro     | En producción, usa siempre el método <code>.sort()</code> de Python, que implementa Timsort (híbrido optimizado en C). |
| 3. Gotcha Crítico   | Elegir siempre el primer elemento como pivote en una lista que ya está ordenada genera $O(n^2)$ .                      |
| 4. Buenas Prácticas | <code>sorted(iterable, key=lambda x: x.propiedad)</code> permite ordenar por cualquier criterio.                       |



### Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.



## Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

| Recurso Oficial         | Descripción                                   | Enlace                                                                    |
|-------------------------|-----------------------------------------------|---------------------------------------------------------------------------|
| Documentación Python 3  | Especificación canónica y biblioteca estándar | <a href="https://docs.python.org/3/">docs.python.org/3/</a>               |
| PEP 8 — Style Guide     | Estándar oficial de formateo y estilo         | <a href="https://peps.python.org/pep-0008/">peps.python.org/pep-0008/</a> |
| Real Python Tutorials   | Patrones de desarrollo e ingeniería           | <a href="https://realpython.com">realpython.com</a>                       |
| Suite wisrovi en GitHub | Paquetes open source de alto rendimiento      | <a href="https://github.com/wisrovi">github.com/wisrovi</a>               |

## 🏆 Desafío Práctico para el Estudiante

### 🎯 Reto de la Semana

Implementa MergeSort y compara el número de comparaciones frente a QuickSort.

### 🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.